

Decaf: Decoupled Dataflows for In Situ Workflows

Orcun Yildiz, Matthieu Dreher, and Tom Peterka

Abstract In situ workflows manage the coordination and communication in a directed graph of heterogeneous tasks executing simultaneously in an high-performance computing system. The communication through the graph can be modeled as a dataflow, and Decaf is a software library for managing the dataflow for in situ workflows. Decaf includes a Python API to define a workflow, creating a complete stand-alone system, but the dataflow design also allows Decaf to support the communication needs of other workflow management systems, because a science campaign may be composed of several workflow tools. Decaf creates efficient parallel communication channels over MPI, including arbitrary data transformations ranging from simple data forwarding to complex data redistribution. Decaf provides three building blocks: (i) a lightweight data model that enables users to define the policies needed to preserve semantic integrity during data redistribution, (ii) flow control designed to prevent overflows in the communication channels between tasks, and (iii) a data contract mechanism that allows users to specify the required data in the parallel communication of the workflow tasks. Decaf has been used in a variety of applications. Two examples are highlighted. The first case is from materials science, where the science campaign consists of several workflow tools that cooperate, and Decaf supports these tools as the dataflow layer. The second problem is motivated by computational cosmology, where the in situ workflow consists of three parallel tasks: synthetic particle generation, Voronoi tessellation, and density estimation.

Orcun Yildiz
Argonne National Laboratory, 9700 S. Cass Ave., Lemont, IL 60439 USA, e-mail: oyildiz@anl.gov

Matthieu Dreher
Formerly of Argonne National Laboratory, e-mail: dreher.matthieu@gmail.com

Tom Peterka
Argonne National Laboratory, 9700 S. Cass Ave., Lemont, IL 60439 USA, e-mail: tpeterka@mcs.anl.gov

1 Introduction

Computational science often involves a workflow of interconnected tasks, several of them being executed on a supercomputer, for example, simulation, analysis, visualization, and user interaction. The workflow can be modeled as a directed graph, which is illustrated in Figure 1, where the nodes of the graph are tasks and the edges represent information exchanged between the tasks. The graph can have cycles where feedback loops exist between tasks so that the result of a task can be used to modify another one (e.g., computational steering).

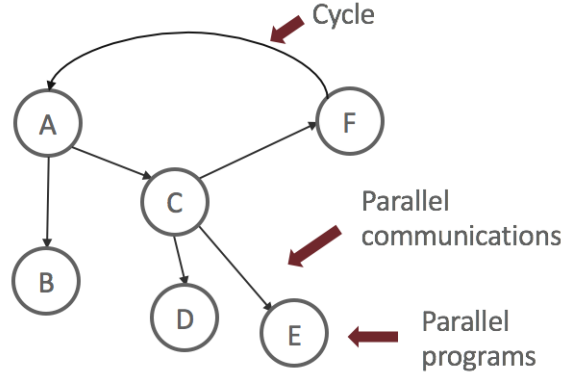


Fig. 1 Workflow graph, where the nodes are tasks and the edges represent information exchanged between the tasks. Tasks are parallel programs such as simulation, analysis, and visualization.

Dataflow is defined as the information exchanged between tasks in a workflow. Traditionally, tasks exchanged data through files. However, the growing mismatch between high-performance computing (HPC) rate and I/O bandwidth motivates shifting from file-oriented workflow models to *in situ workflows*, where dataflows are through memory or the supercomputer interconnect, avoiding the storage I/O bottleneck.

Decaf is a dataflow library for *in situ* workflows. Figure 2 shows an overview of Decaf and its components. Decaf is designed as a library separate from the workflow engine because a science campaign may consist of several workflow tools that need to cooperate, and this design allows one dataflow library to support multiple workflow tools simultaneously. However, several challenges need to be addressed as a result of this design choice. The dataflow needs to manage the data exchange between heterogeneous programming and data models because simulation and analysis tasks are often developed independently. Workflows can be instantiated with different numbers of resources (e.g., MPI ranks) per task, and the dataflow has to transform and redistribute data during the exchange between tasks. The dataflow runtime needs to support any generic directed graph because workflow graphs come in different shapes and sizes, ranging from a simple pipeline of two or three tasks to more

complex topologies including fan-in, fan-out, and cycles. To meet these challenges, Decaf is equipped with the following features:

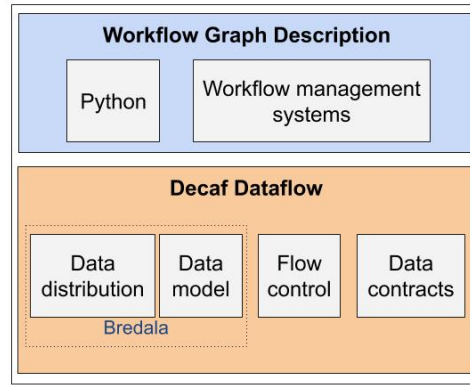


Fig. 2 Overview of Decaf dataflow library and its components.

- A decoupled dataflow, a hybrid of tight and loose coupling of tasks through an optional intermediate set of resources that can be used to transform or redistribute data between tasks
- A flow control library designed to manage disparate data rates between tasks
- A data contract mechanism that allows users to specify the required data in the parallel communication of the tasks
- A high-level Python API for the workflow graph definition so that Decaf can be used as a stand-alone workflow system when it is not used in conjunction with other workflow tools
- A design that allows composition of different workflow systems so that the workflow can be defined in a completely different software tool with Decaf providing only the dataflow capability
- A message-driven runtime for executing workflow tasks when all messages for a task have been received, which supports any directed graph topology, including cycles
- Support for the MPMD (multiple program multiple data) capability of MPI to run multiple executables in the same job launch, a feature available on most supercomputing and cluster platforms

The remainder of this chapter is organized as follows. Section 2 gives background for in situ workflows and presents related work. Section 3 describes Decaf's design and key features. Section 4 illustrates the capabilities of Decaf. Section 5 gives a summary and briefly describes future work.

2 Background and Related Work

This section first provides a brief background on types of in situ workflows with respect to the coupling mode between the workflow tasks. Then, it presents related work by categorizing the workflows according to their coupling modes.

2.1 Types of In Situ Workflows

Figure 3 illustrates the different in situ coupling modes when two tasks are connected in the workflow graph. In time division, the two tasks, producer and consumer, share the same resources and run sequentially in time. For example, a simulation (producer) and analysis (consumer) task may take turns operating on the same time step of data. The simulation computes the data for one time step and then waits while the analysis task processes the data, which then waits while the simulation generates the next step.

In space division, the producer and consumer run on different resources concurrently in time. Using the same simulation-analysis example, the simulation computes the first time step, copies or sends it to the consumer, and then computes the second time step while the analysis code processes the first time step. Both modes are used in practice, and there are time-space tradeoffs between them.

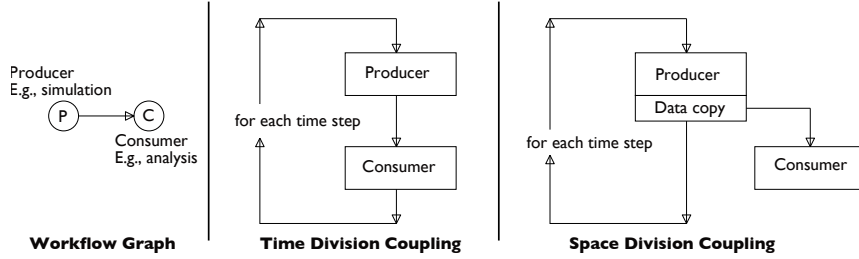


Fig. 3 Time division and space division in situ coupling of a producer and consumer task [17]. In time division coupling, these tasks execute sequentially on the same resource; in space division coupling, they execute concurrently on separate resources.

2.2 In Situ Workflow Runtimes

In situ tools have been developed by various communities with performance, programmability, and portability tradeoffs highlighted by Dorier et al. [13]. In this section, these tools are characterized according to the type of coupling between their tasks.

2.2.1 Time Division

Scientific visualization is one of the main targets for in situ analytics, motivated by the need for high bandwidth and temporal resolution. Yu et al. [35] integrated a particle volume renderer synchronously in a combustion code (S3D). The renderer processing required 10% of the total execution time to produce images. Current production visualization tools include ParaView [3] and VisIt [2] for postprocessing. Both have an in situ library, respectively Catalyst [21] and Libsim [34], to process simulation data. The goal of these libraries is to convert the internal data structures of the simulation to VTK data structures. The libraries then execute the rendering pipeline, usually synchronously, although other configurations are possible.

2.2.2 Space Division

Other in situ tools originate in the I/O community. Originally designed for I/O staging, these tools have subsequently coupled analysis and visualization applications together with simulations. ADIOS [26] is a common interface for multiple I/O methods. Initially developed to store data efficiently in parallel, it now provides methods to share data between codes and transform data in situ along the I/O path [6].

Several frameworks using the ADIOS interface can transfer data asynchronously to dedicated resources. DataStager [1] can schedule data movement while the simulation is in its computational phase, avoiding unnecessary network contention. Flex-Path [8] provides a publisher/subscriber model to exchange data between parallel codes having different numbers of processes ($M \times N$ communication pattern).

DataSpaces [9] implements a distributed shared-memory space with a publisher/subscriber interface for external applications. DataSpaces indexes data based on a space-filling curve. Data are then distributed among data servers based on their index. The index is used both for pushing data into DataSpaces and for retrieving data efficiently from it.

Damaris [14, 11] splits the global MPI communicator of a simulation to dedicated cores or nodes [12] in order to run the analysis concurrently with the application. The framework was used to stage I/O and for in situ visualizations. Functional partitioning [25] also uses dedicated cores for I/O with a FUSE interface.

HDF5 DSM [5] connects a simulation code to ParaView by using the HDF5 interface. The simulation and the ParaView servers can run in separate jobs. Both jobs read and write data to the shared virtual HDF5 file layer; steering is also possible.

2.2.3 Hybrid Approaches

Time division and space division each have pros and cons, motivating flexibility in analytics placement [36]. Several solutions now support both time and space division. FlexIO [37] allows time division tasks on the same compute cores or

space division on dedicated cores. Bennett et al. [4] combined computations in time division followed by other processing on dedicated nodes using DART servers [10] to transfer data, applying several analysis algorithms to the combustion code S3D.

FlowVR [19] couples parallel codes to form a graph of tasks. Each task is usually a separate executable. Communication between parallel codes is managed by a daemon running on each node. Decaf also adopts a hybrid approach. Similar to FlowVR, Decaf composes multiple executables to form a workflow. However, its execution model is designed for in situ HPC environments, and it does not rely on a separate daemon to manage the graph execution.

3 Design

This section provides an overview of Decaf and its main components. Decaf’s main building block is a dataflow, which consists of a producer, a consumer, and a communication object. Decaf implementation is defined in two parts. First, the user defines the dataflow using Decaf’s Python scripting interface, which produces a JSON file describing the layout. Second, the user implements the producer and consumer of that dataflow using Decaf’s C/C++ API. Besides the dataflow component, Decaf provides three building blocks: (i) a data model to preserve semantic integrity during data redistribution, (ii) a flow control to manage disparate rates between tasks, and (iii) a data contract mechanism to specify the required data in the parallel communication of the tasks.

3.1 Decaf Dataflow

A dataflow is the association of a producer, a consumer, and a communication object to exchange data between the producer and consumer. Producers and consumers, called nodes in Decaf, are parallel codes such as simulation and analytics. For instance, Gromacs [33], a molecular dynamics simulation, typically generates atom positions, and scientists use tools such as VMD [24] to visualize and analyze the atom positions. In an in situ context, Gromacs and VMD communicate directly through memory. Data structures between the two codes are often different, and some data transformation might be required. This can be as simple as converting data units (from nanometers to angstroms) to more complex data rearrangements such as mapping a simulation data structure to a VMD data structure.

These data transformations can require extensive computations. To address these needs, Decaf dataflow includes an intermediate staging area with computational resources. This staging area is called a *link*, which is an intermediate parallel program transforming data between a producer and a consumer.

Figure 4 shows the organization of a simple dataflow from producer to link to consumer. Advanced workflow graphs can be obtained by combining multiple such

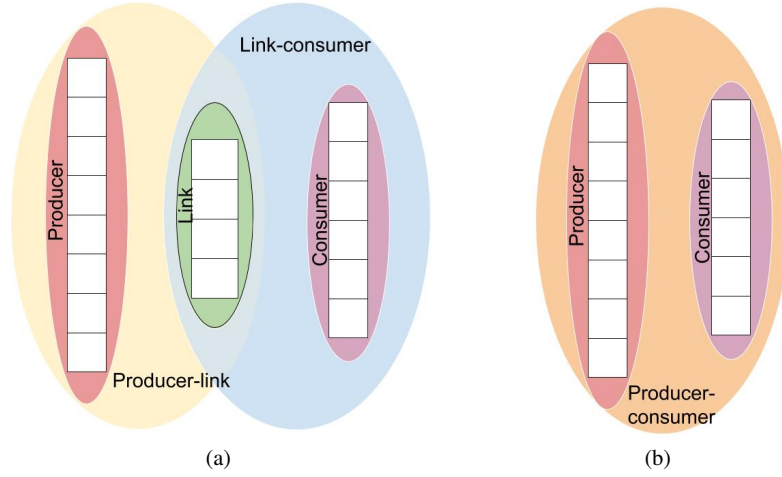


Fig. 4 (a) Decaf forms 5 communicators for a dataflow: one communicator each for producer, consumer, and link, plus two more communicators for the overlap between producer-link and link-consumer. (b) The link is optional; when not used, there are only 3 communicators.

dataflows. The producer, link, and consumer are all MPI programs, each with its own MPI communicator. The dataflow creates two additional communicators: producer-link and link-consumer. Section 3.2 explains how to describe such graphs and how the runtime executes them. Section 3.4 describes how data are exchanged through the communicators.

The link resources are optional; one can disable the link when no data manipulations are required between the producer and consumer. In that case, the user does not assign any resources to the link, and the runtime creates only one communicator directly between the producer and the consumer.

3.2 Workflow Graph Description and Runtime Execution

A Decaf workflow is the composition of multiple dataflows. For instance, to create a pipeline of three tasks, the user declares two dataflows, where the second task is both consumer in the first dataflow and producer in the second dataflow. Decaf does not apply any constraints on the graph topology. In particular, the user can define cycles in the graph for computational steering.

The user describes the workflow graph in a Python script. Listing 1 presents the code generating a three-task pipeline. First, the user describes individual tasks with a name and a set of resources (MPI ranks). Second, the user describes the dataflow linking two tasks (producer, consumer), a set of resources, and the choice of the redistribution strategy. The same task can be used by several dataflows as producer

```

w = nx.DiGraph()

# Task declaration
w.add_node('node0', start_proc=0, nprocs=4)
w.add_node('node1', start_proc=7, nprocs=2)
w.add_node('node2', start_proc=11, nprocs=1)

# Dataflow declaration
w.add_edge('node0', 'node1', start_proc=4, nprocs=3,
           func='dflow', path=mod_path,
           prod_dflow_redist='count',
           dflow_con_redist='count')
w.add_edge('node1', 'node2', start_proc=9, nprocs=2,
           func='dflow', path=mod_path,
           prod_dflow_redist='count',
           dflow_con_redist='count')

wf.workflowToJson(w, mod_path, "linear3.json")

```

Listing 1 Code sample to generate a 3-node pipeline.

or consumer. Once all the tasks are described, the user calls `workflowToJson` to generate a JSON file. That file contains an intermediate representation of the workflow read by the Decaf runtime upon initialization. The graph description is done prior to the launch of the workflow.

In a Decaf workflow, every task is an MPI program. Each task can be a separate program, or all tasks may be combined in a single program. Decaf relies on the MPMD capability of MPI to launch all the programs. With this method, `MPI_COMM_WORLD` is shared among all the executables. Based on the information provided by the JSON file, Decaf creates either three or five communicators per dataflow (Figure 4). This method requires existing codes such as simulations only to replace their equivalent of `MPI_COMM_WORLD` by the communicator provided by Decaf.

3.3 Structure of Task Code

To minimize the required code modifications for integration of user codes, Decaf provides a simple put/get API that allows tasks to send/receive data to/from the rest of the workflow. Listing 2 presents a typical program.

After the initialization of the runtime (Section 3.2), the code loops over incoming data. The `get(in_data)` call waits until data are received on all inbound links or a termination message is received, whereby the code exits the loop. Incoming data are processed and sent by calling `put(out_data)`. The `terminate()` call signals to the runtime to exit the application. This API is available in both C and C++.

```

int main(int argc,
        char** argv)
{
    // define the workflow
    Workflow workflow;
    make_wflow_from_json(workflow, "linear3.json");

    // initialize MPI and Decaf
    MPI_Init(NULL, NULL);
    Decaf* decaf = new Decaf(MPI_COMM_WORLD, workflow);

    // run the task
    vector<pConstructData> in_data;
    while (decaf->get(in_data))
    {
        // process the input data

        // send the results outbound
        pConstructData out_data;
        decaf->put(out_data);
    }
    // end the task
    decaf->terminate();

    // cleanup
    delete decaf;
    MPI_Finalize();

    return 0;
}

```

Listing 2 A typical program constructs the Decaf object and executes a node task. The task waits for all its inputs to be satisfied and then processes the received data. It can safely do so in an infinite loop because the Decaf `get()` function returns false upon termination.

3.4 Data Model and Data Redistribution in the Dataflow

The key question is how to maintain a valid data model that preserves the original semantics when data are redistributed between M producer and N consumer processes. One major challenge is that none of these processes has a complete view of the data decomposition; hence, considerable effort is required to match decomposition between producer and consumer. Also, different scenarios require different logic, and extra hints from the user can simplify the required computation. For example, if only one producer rank has a particular dataset and only one consumer rank wants to read it, then the underlying dataflow must reconcile this information. With this goal in mind, Bredala [16] is a library designed to build a data model with enough information to preserve the semantics of data during data redistribution in in situ dataflows.

Decaf uses both the data model and the redistribution components of Bredala. The data model serves as the data interface to exchange data between tasks within the workflow. For each dataflow, Decaf creates two redistribution components: one between the producer and the link and one between the link and the consumer. When calling `put()`, Decaf passes the data to the corresponding redistribution component that manipulates and transmits the data to its destination. When calling `get()`, Decaf receives data on the corresponding redistribution component and transmits it to the

```

int gid;
float pos[3*Nb_Pos];

// Declaring the fields
SimpleFieldId fieldGid(gid);
// One semantic item consists of 3 floats (e.g., position in a 3D space)
ArrayFieldId fieldPos(pos, 3*Nb_Pos, 3);

pConstructData container;
container->appendData("gid", fieldGid,
                     DECAF_NOFLAG, DECAF_SHARED,
                     DECAF_SPLIT_DEFAULT,
                     DECAF_MERGE_KEEP_FIRST);
container->appendData("pos", fieldPos,
                     DECAF_POS, DECAF_PRIVATE,
                     DECAF_SPLIT_DEFAULT,
                     DECAF_MERGE_APPEND);

```

Listing 3 Code sample to generate a simple data model. The data model is composed of a global identifier and an array of positions.

task. These data model and redistribution components are detailed in the following subsections.

3.4.1 Data Model Description

Bredala protects the semantic integrity of a data model during split and merge operations using the notion of a semantic item. This is the smallest subset of data that contains all the fields of the original data structure while preserving its semantics. In Bredala, the split and merge functions are embedded within the data model. These functions divide a data model into subdata models and merge subdata models into a larger data model safely.

Listing 3 gives an example to describe a simple data model with a global identifier and an array of positions. First, each field is declared with its type (e.g., Simple/Array) and its semantic information if any. For instance, the last argument in the *fieldPos* declaration indicates that this position is composed of three floating points. This semantic information will be used by Bredala for preserving the semantics of this field during data redistribution. Then, each field is pushed in a container (*pConstructData*). Besides the name and the field variables, the user provides a semantic flag, a visibility flag, and flags for split and merge policies.

The semantic flag indicates whether a field has extra semantic meaning in the data model. For instance, the flag *DECAF_POS* indicates that the field *pos* represents the positions of the semantic items. Then, this information can be used to redistribute data spatially. The visibility flag indicates whether the value of the field is shared by all the items in the data model (e.g., a global identifier with *DECAF_SHARED* flag) or when this value is different for each group (e.g., different position values with *DECAF_PRIVATE* flag). The split and merge flags tell Bredala which method to use during the split and merge operations. We will see an example of this in the next section.

3.4.2 Data Redistribution

Bredala provides redistribution components to transfer a data model from M to N processes. The components use the semantic information provided by the data model to extract individual semantic items and send them to the appropriate destination. Bredala provides a safe way to access, extract, and merge semantic items within a data model using three redistribution strategies: round-robin, contiguous, and block redistribution (Figure 5). The round-robin strategy performs the data redistribution item by item in a round-robin fashion. The contiguous strategy redistributes all the items while preserving their order. The block redistribution strategy divides a global domain into subdomains and attributes each item to a particular subdomain.

In each redistribution strategy, data redistribution consists of three main steps: (1) splitting the data model on the producer side; (2) transferring these data models between M producer and N consumer processes over MPI; and (3) merging the received data models on N consumer processes. Splitting and merging are performed based on the user-defined policies in the data model. For instance, the DECAF_SPLIT_DEFAULT policy in Listing 3 splits the array of positions in chunks of three floating-point values based on the semantics of the field *pos*. The policy MERGE_APPEND_VALUES concatenates the two arrays of position values.

3.5 Flow Control Library

In space division mode, workflow tasks run independently on dedicated resources and exchange data asynchronously. However, tasks may run at different rates. For instance, a consumer might not be able to process the incoming data from a producer fast enough, overflowing or stalling the dataflow between producer and consumer. To mitigate such scenarios, Decaf provides a flow control library, Manala [20], to control the flow of messages between producers and consumers.

Figure 6 illustrates the operation of Manala, which manages the message queue between producers and consumers. First, Manala intercepts the messages from producers and stores them in a storage hierarchy consisting of main memory, shared remote memory (i.e., distributed in-memory key/value store), and the file system. The hierarchical design allows Manala to buffer as many messages as possible before blocking producers. Then, Manala selects the messages that need to be forwarded to the consumers depending on the flow control policy defined by the user. Manala provides four different flow control policies: *all*, *some*, *latest*, and *hybrid*. The *all* policy applies FIFO flow control, Decaf’s default policy when Manala is not used, forwarding all messages to consumers in the order they have been sent by producers. The *some* policy also preserves the order of the messages, but this time Manala stores one out of every x messages received and drops the rest of them. In the *latest* policy, Manala forwards the most recent data to the consumers and drops the older data. The *hybrid* policy combines the *some* and *latest* policies by forwarding one out of every x messages received among the most recent data. These different policies

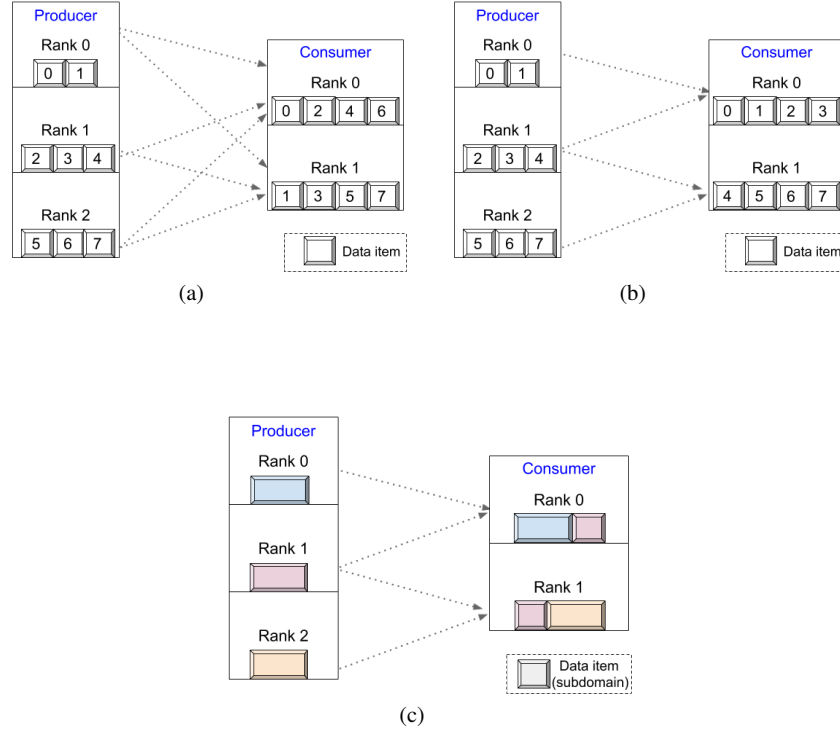


Fig. 5 Redistribution strategies available in Bredala. (a) The round-robin strategy performs the data redistribution item by item in a round-robin fashion. (b) The contiguous strategy redistributes all the items while preserving their order. (c) The block redistribution strategy divides a global domain into subdomains and attributes each item to a particular subdomain.

provide flexibility in managing the flow of messages in the workflow in order to meet application requirements.

3.6 Data Contract Mechanism

Decaf decouples data management between producers and consumers by handling the exchange of messages between these tasks. Traditionally, the data to be sent to different consumers are usually hard-coded in producers. This requires modifying the producer code every time the consumer changes, which is inefficient. To address this problem, Decaf provides a mechanism [27] where users can specify a data contract between a producer and consumer of data. This data contract mechanism allows

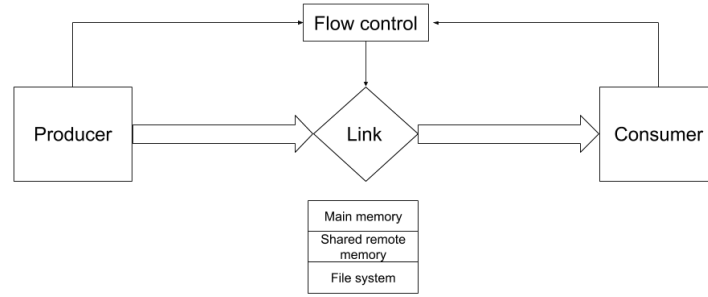


Fig. 6 Manala, a flexible flow control library, controls the flow of messages between the producer and the consumer by using a storage hierarchy consisting of main memory, shared remote memory, and the file system.

the Decaf runtime to automatically extract the data from producers depending on the requirements of the consumers, minimizing required code changes. Moreover, the same producer can be coupled to different consumers with different data requirements, without specializing the producer task for each consumer. Without data contracts, a single producer would have to send the superset of data required by all the consumers. Doing so, however, would create redundant data transfers over the network, wasting resources.

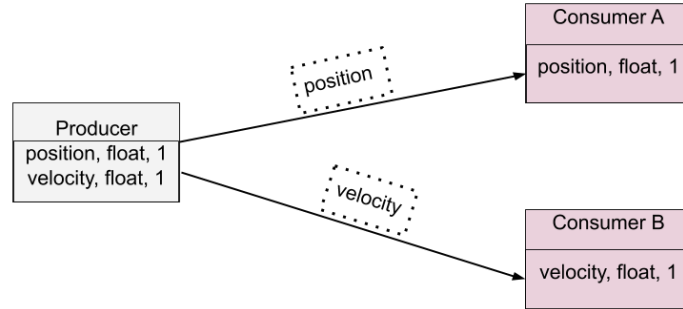


Fig. 7 Sample data contract mechanism between one producer and two different consumers. The dashed-line rectangles represent the data fields forwarded in each dataflow.

In the contract, the data model is represented as a set of fields. The field is a triplet, *(name, type, periodicity)*, where periodicity is the frequency at which the producer

```

# Contract declaration in workflow tasks
...
producer.addOutput('position', 'float', 1)
producer.addOutput('velocity', 'float', 1)
...
consumerA.addInput('position', 'float', 1)
consumerB.addInput('velocity', 'float', 1)

# Contract declaration in dataflows
...
edgeA.addInput('position', 'float', 1)
edgeA.addOutput('position', 'float', 1)
edgeA.setForwardFields(True)
...
edgeB.addInput('velocity', 'float', 1)
edgeB.addOutput('velocity', 'float', 1)
edgeB.setForwardFields(True)

```

Listing 4 Required modifications in the Python API for the declaration of data contracts.

generates the field. The user specifies the set of fields that the producer makes available for output, as well as the data fields needed for input to the consumer. Then, Decaf’s data contract mechanism generates the matching list to deduce which data fields need to be forwarded in the dataflow between the producer and the consumer. Figure 7 illustrates a simple data contract example between one producer and two different consumers. Listing 4 shows the necessary additions to describe such a contract in the workflow graph description.

4 Science Drivers

Two science use cases are highlighted below: molecular dynamics and cosmology. In the molecular dynamics example, the steering capabilities of Decaf and its interoperability with other in situ libraries to create heterogeneous workflows are illustrated. In the cosmological example, the scalability of Decaf is tested, and its capacity to handle large, complex data structures is demonstrated.

4.1 Molecular Dynamics

The first example is steering a molecular dynamics simulation to trigger a biological mechanism. In the FepA protein, a channel in the periplasm of Gram-negative bacteria provides an entryway where compounds can traverse the membrane of the cell. This mechanism is of interest to biologists because drugs pass through these channels, and steering the simulation allows a biologist to push a complex within a channel and accelerate the traversal process.

Workflows supporting computational steering pose several challenges. First, a steering workflow needs to support cycles in the workflow graph, which can po-

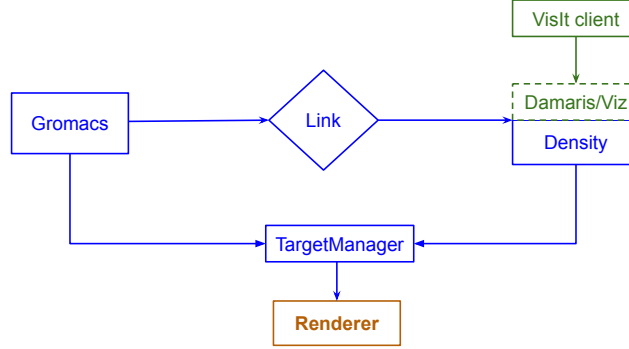


Fig. 8 Steering workflow with Decaf (blue), FlowVR (brown), and Damaris/Viz connected to the VisIt client (green). The steering part is managed by Decaf while visualizations are handled by Damaris and FlowVR. Decaf and FlowVR tasks and communications are in space division mode (blue arrows) while Damaris execution is executed in time division mode (green arrow) during the execution of *Density*.

tentially generate deadlocks. Second, the steering process must be performed asynchronously to avoid blocking the simulation. Third, because multiple types of interactions may be needed (force feedback, visualization, density computation), Decaf must be able to integrate different tools from different communities into a single workflow.

In this experiment, a steering workflow is implemented with Decaf to guide the iron complex toward the FepA channel, based on the implementation proposed by Dreher et al. [18]. Figure 8 summarizes the workflow. First, Decaf API calls are integrated into Gromacs [33], a molecular dynamics simulation code, to add external forces and expose atom positions to the rest of the workflow. Second, in a link (*LinkMorton*), a Morton index [30] is computed for each atom particle. Third, a 3D density grid (*Density*) is computed. Fourth, the force (*TargetManager*) to guide the system is computed. The trajectory for the complex to follow is defined by a list of target positions, provided by the user, forming a path. A path-finding algorithm [23] based on the previously computed 3D density grid guides the iron complex from one target to the next.

At each step of the workflow, the fields of the data model are modified. First, the atom positions and their indices from the simulation are sent to the link. Second, the Morton indices are added, and data are reorganized to compute the density grid. Third, the Morton indices are removed from the data model (no longer necessary), and a 3D grid is appended to the data model. Then, the result is sent to *TargetManager*, which broadcasts forces to the simulation.

For this application, FlowVR and Damaris/Viz [14] are used to perform in situ visualization. First, the workflow is connected to the renderer of Dreher et al. [18] based on Hyperballs [7] to visualize the molecular model and the state of the steering system (Figure 9(a)). This visualization is preferred by biologists to guide the

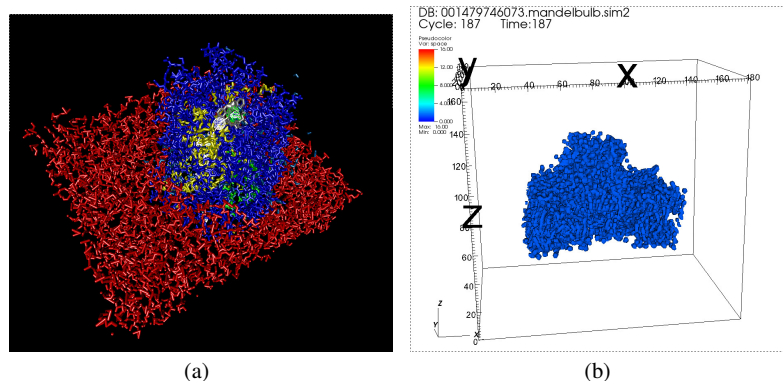


Fig. 9 (a) Visualization of the molecular system with FlowVR performed in space division mode, as depicted in Figure 8. (b) Visualization of the density grid with Damaris/Viz using VisIt in time division mode.

complex because the user can navigate within the molecular structure and track the iron complex. This visualization is performed in space division mode. Second, Damaris/Viz is used in time division mode to visualize the density grid with VisIt (Figure 9(b)). The visualization of the density grid shows the biologists where low-density areas are located, which are good candidates for the iron complex to go through. Since no tools support both visualizations, several visualization packages had to be combined in the same workflow. Without Decaf, the biologists would normally perform such visualizations separately, resulting in redundant data transfers among these tools and loss of productivity.

The following tests are conducted on Froggy, a 138-node cluster from the Ciment infrastructure. Each compute node is equipped with 2 eight-core processors, Sandy Bridge-EP E5-2670 at 2.6 GHz with 64 GB of memory. Nodes are interconnected through an FDR InfiniBand network. FlowVR 2.1 and Gromacs 4.6 are compiled with Intel MPI 4.1.0. For all experiments, the molecular model of the FepA is used, which is composed of about 70,000 atoms. For all scenarios, the atom positions are extracted from the simulation every 100 iterations.

An experiment was conducted to study the impact of the steering pipeline on Gromacs performance. The workflow is created as described in Figure 8. Each task runs in space division mode, where one node behaves as a staging node, and the remaining nodes are simulation nodes. The Density (four cores) and TargetManager (one core) tasks are hosted on one node. The Density task is limited to four cores since the density grid is relatively small (170x80x180). Gromacs runs on each simulation node using 15 cores per node. The LinkMorton task runs on the remaining core on each simulation node to preserve the data locality. MPI is configured to bind each MPI rank to a given core in order to avoid process migration and reduce performance variability.

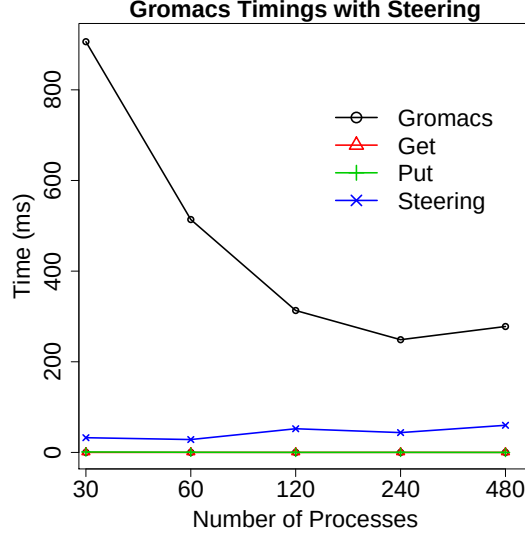


Fig. 10 Time decomposition of Gromacs performance modified with Decaf and connected to the steering pipeline without visualization. The measured time is the average over a full I/O cycle (100 iterations).

Figure 10 presents the timing of the Gromacs execution and the steering process for 100 simulation iterations. **Steering** indicates the accumulated execution time of the **LinkMorton**, **Density**, and **TargetManager** tasks. The steering process has a time budget of 100 simulation iterations to complete before blocking the simulation. At 480 cores, the simulation requires on average 277 ms to compute 100 iterations. In all cases, the time spent by Gromacs in Decaf operations represents less than 0.1% of the allocated time budget.

4.2 Cosmology

The second use case is the analysis of dark matter tracer particles from a Vlasov-Poisson N-body cosmology code. This workflow converts particle positions to the deposition of particle density onto a regular 2D and 3D grid, using a Voronoi tessellation as an intermediate step. For high dynamic range data such as dark matter particles, computing the Voronoi tessellation first produces more accurate density estimates than less expensive methods that compute the density directly from the particle data [31]. The data model produced by the cosmology code, a set of particles, is transformed into the intermediate data model of the Voronoi tessellation, an unstructured polyhedral mesh, before being converted into a 2D or 3D regular grid of density scalar values.

The N-body cosmology code is HACC [22]. The tessellation and density estimator codes are built on the Tess library [32], which in turn is built on the DIY data-parallel programming model [28]. The three tasks are coupled with a link between each task in a 3-node linear workflow graph using Decaf. Decaf enables exchange of complex data structures among these tasks, transparent to such task codes. Normally, this would be done manually by the user by performing the data transformation for each particular data type in the task code. For instance, the link between the simulation and tessellation nodes rearranges the particles from the structure of arrays format produced by HACC to an array of structures format required by Tess.

All three tasks are parallel MPI programs that scale to large numbers of MPI processes. The intermediate tessellation has a large memory footprint, approximately 15 times as large as the simulation data, making it necessary to compute the tessellation on a separate set of compute nodes from the simulation or density estimation. The density image (Figure 11) is a fraction of the simulation data size; but because of the large memory footprints of the analysis tasks and the desire to perform analysis simultaneously with simulation, all the nodes and links are configured in space division mode to use separate resources. In the following experiment, all tasks and links have the same number of processes.

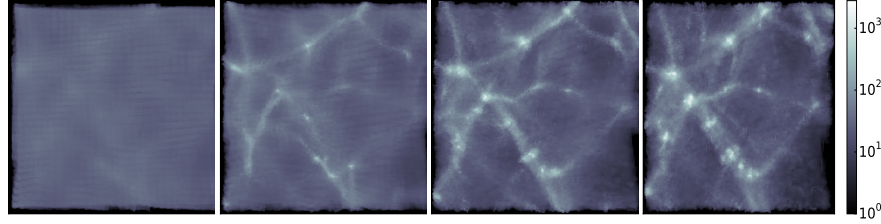


Fig. 11 (Left to right) Output of cosmology analysis workflow at time steps 10, 30, 60, and 100.

The following tests were conducted on the IBM Blue Gene/Q Vesta machine at the Argonne Leadership Computing Facility at Argonne National Laboratory. Vesta is a testing and development platform consisting of 2K nodes, each node with 16 cores (PowerPC A2 1.6 GHz), 16 GB of RAM, and 64 hardware threads. Eight MPI processes per compute node are used. The Clang compiler, based on LLVM version 3.9, is used to compile the code with -O3 optimization.

Figure 12 shows the performance of a strong-scaling test in log-log scale. In this test, there are 128^3 particles estimated onto a 512^2 2D output grid. The vertical axis is the time to simulate a total of 100 time steps and to compute the tessellation and density estimation every 10 time steps. The horizontal axis is the total number of MPI processes for the entire workflow. The curves in the plot include total (end-to-end) time as well as the time for each of the three tasks.

The overall strong-scaling efficiency is approximately 50% from 40 to 320 processes. After 320 processes, the diminishing number of particles per process combined with the increasing imbalance between processes reduces the scalability of

the tessellation, which in turn affects the rest of the workflow. For example, at 1,280 processes, the final time step of the simulation produces only 16 particles in the least-loaded process, and 1,972 particles in the most-loaded one. The load imbalance as dark matter particles cluster into halos is expected; using a k-d tree instead of a regular grid of blocks to perform the tessellation [29] can mitigate this situation.

The tasks overlap in time (space division) so that the total time is only slightly longer than that of the longest task, which is density estimation, but much less than the sum of the parts. This is especially true over the total of multiple time steps, effectively hiding the analysis time from the simulation.

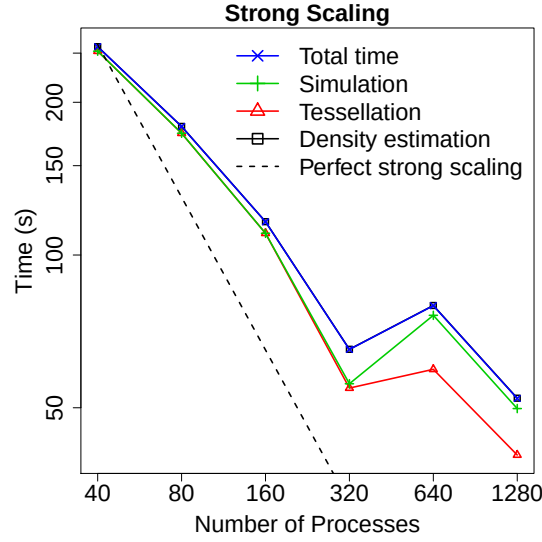


Fig. 12 Strong scaling of the cosmology simulation-tessellation-density estimation workflow. Good scaling efficiency is shown until the small number of particles per process and load imbalance of the tessellation reduce efficiency. The time of the analysis tasks is effectively overlapped with the simulation time.

5 Conclusion

This chapter introduces Decaf, middleware for coupling in situ tasks to form workflows. Decaf makes it easy to modify existing simulation or analytics codes with its simple API. It is also simple to describe the workflow in Python as a directed graph where nodes are tasks and edges are communication channels. Decaf does not impose any constraints on the graph topology and can manage graphs with cycles. The Decaf runtime handles building the communication channels and the data exchanges. Decaf protects the semantic integrity of data during parallel redistribution with its

lightweight data model. Decaf’s data contract mechanism allows the Decaf runtime to automatically extract the data from producers depending on the requirements of consumers. Decaf is also equipped with a flow control library to manage disparate rates between tasks.

Decaf’s capabilities are demonstrated with science use cases. The first is a steering scenario merging a molecular dynamics simulation, user analytic codes, and visualizations with FlowVR and Damaris/Viz workflow tools. In the second example, Decaf’s scalability up to 1,280 cores is studied with the analysis of dark matter that involves the exchange of complex data structures.

Currently, Decaf uses a static Python configuration file to define the workflow. In future work, bringing elasticity to Decaf will be explored, since resources may need to be redistributed among the tasks of the graph at runtime to accommodate for changes in requirements [15].

The Decaf project is available in open source.¹

Acknowledgements This work is supported by Advanced Scientific Computing Research, Office of Science, U.S. Department of Energy, under Contract DE-AC02-06CH11357, program manager Laura Biven. This research used resources of the Argonne Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC02-06CH11357.

References

1. Abbasi, H., Wolf, M., Eisenhauer, G., Klasky, S., Schwan, K., Zheng, F.: Datastager: Scalable Data Staging Services for Petascale Applications. In: Proceedings of the 18th ACM International Symposium on High Performance Distributed Computing, HPDC ’09, pp. 39–48. ACM, New York, NY, USA (2009). DOI 10.1145/1551609.1551618. URL <http://doi.acm.org/10.1145/1551609.1551618>
2. Ahern, S., Brugger, E., Whitlock, B., Meredith, J.S., Biagas, K., Miller, M.C., Childs, H.: Visit: Experiences with sustainable software. arXiv preprint arXiv:1309.1796 (2013)
3. Ahrens, J., Geveci, B., Law, C.: 36 paraview: An end-user tool for large-data visualization. *The Visualization Handbook* p. 717 (2005)
4. Bennett, J., Abbasi, H., Bremer, P.T., Grout, R., Gyulassy, A., Jin, T., Klasky, S., Kolla, H., Parashar, M., Pascucci, V., Pebay, P., Thompson, D., Yu, H., Zhang, F., Chen, J.: Combining in-situ and in-transit processing to enable extreme-scale scientific analysis. In: High Performance Computing, Networking, Storage and Analysis (SC), 2012 International Conference for, pp. 1–9 (2012). DOI 10.1109/SC.2012.31
5. Biddiscombe, J., Soumagne, J., Oger, G., Guibert, D., Piccinalli, J.G.: Parallel Computational Steering and Analysis for HPC Applications using a ParaView Interface and the HDF5 DSM Virtual File Driver. In: T. Kuhlen, R. Pajarola, K. Zhou (eds.) Eurographics Symposium on Parallel Graphics and Visualization. The Eurographics Association (2011). DOI 10.2312/EGPGV/EGPGV11/091-100
6. Boyuka, D., Lakshminarasimham, S., Zou, X., Gong, Z., Jenkins, J., Schendel, E., Podhorszki, N., Liu, Q., Klasky, S., Samatova, N.: Transparent In Situ Data Transformations in ADIOS. In: Cluster, Cloud and Grid Computing (CCGrid), 2014 14th IEEE/ACM International Symposium on, pp. 256–266 (2014). DOI 10.1109/CCGrid.2014.73

¹ <https://github.com/tpeterka/decaf>

7. Chavent, M., Vanel, A., Tek, A., Levy, B., Robert, S., Raffin, B., Baaden, M.: GPU-accelerated atom and dynamic bond visualization using hyperballs: A unified algorithm for balls, sticks, and hyperboloids. *Journal of Computational Chemistry* **32**(13), 2924–2935 (2011)
8. Dayal, J., Bratcher, D., Eisenhauer, G., Schwan, K., Wolf, M., Zhang, X., Abbasi, H., Klasky, S., Podhorszki, N.: Flexpath: Type-based publish/subscribe system for large-scale science analytics. In: *Cluster, Cloud and Grid Computing (CCGrid)*, 2014 14th IEEE/ACM International Symposium on, pp. 246–255 (2014). DOI 10.1109/CCGrid.2014.104
9. Docan, C., Parashar, M., Klasky, S.: DataSpaces: an Interaction and Coordination Framework for Coupled Simulation Workflows. In: *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing (HPDC '10)*, pp. 25–36. ACM, New York, NY, USA (2010). DOI 10.1145/1851476.1851481
10. Docan, C., Parashar, M., Klasky, S.: Enabling high-speed asynchronous data extraction and transfer using dart. *Concurrency and Computation: Practice and Experience* **22**(9), 1181–1204 (2010). DOI 10.1002/cpe.1567. URL <http://dx.doi.org/10.1002/cpe.1567>
11. Dorier, M., Antoniu, G., Cappello, F., Snir, M., Orf, L.: Damaris: How to Efficiently Leverage Multicore Parallelism to Achieve Scalable, Jitter-free I/O. In: *CLUSTER - IEEE International Conference on Cluster Computing*. IEEE (2012)
12. Dorier, M., Antoniu, G., Cappello, F., Snir, M., Sisneros, R., Yildiz, O., Ibrahim, S., Peterka, T., Orf, L.: Damaris: Addressing Performance Variability in Data Management for Post-Petascale Simulations. *ACM Transactions on Parallel Computing (ToPC)* (2016)
13. Dorier, M., Dreher, M., Peterka, T., Wozniak, J.M., Antoniu, G., Raffin, B.: Lessons learned from building in situ coupling frameworks. In: *Proceedings of the First Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*, pp. 19–24. ACM (2015)
14. Dorier, M., Sisneros Roberto, R., Peterka, T., Antoniu, G., Semeraro Dave, B.: Damaris/Viz: a Nonintrusive, Adaptable and User-Friendly In Situ Visualization Framework. In: *LDAV - IEEE Symposium on Large-Scale Data Analysis and Visualization*. Atlanta, United States (2013). URL <https://hal.inria.fr/hal-00859603>
15. Dorier, M., Yildiz, O., Peterka, T., Ross, R.: The challenges of elastic in situ analysis and visualization. In: *Proceedings of the Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*, pp. 23–28 (2019)
16. Dreher, M., Peterka, T.: Bredala: Semantic Data Redistribution for In Situ Applications. In: *CLUSTER - IEEE International Conference on Cluster Computing*. IEEE (2016)
17. Dreher, M., Peterka, T.: Decaf: Decoupled dataflows for in situ high-performance workflows. Tech. rep., Argonne National Lab.(ANL), Argonne, IL (United States) (2017)
18. Dreher, M., Prevotau-Jonquet, J., Trellet, M., Piuze, M., Baaden, M., Raffin, B., Férey, N., Robert, S., Limet, S.: ExaViz: a Flexible Framework to Analyse, Steer and Interact with Molecular Dynamics Simulations. *Faraday Discussions of the Chemical Society* **169**, 119–142 (2014). DOI 10.1039/C3FD00142C. URL <https://hal.inria.fr/hal-00942627>
19. Dreher, M., Raffin, B.: A Flexible Framework for Asynchronous In Situ and In Transit Analytics for Scientific Simulations. In: *14th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. Chicago, United States (2014). URL <https://hal.inria.fr/hal-00941413>
20. Dreher, M., Sasikumar, K., Sankaranarayanan, S., Peterka, T.: Manala: A flexible flow control library for asynchronous task communication. In: *2017 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 509–519. IEEE (2017)
21. Fabian, N., Moreland, K., Thompson, D., Bauer, A., Marion, P., Geveci, B., Rasquin, M., Jansen, K.: The Paraview Coprocessing Library: a Scalable, General Purpose In Situ Visualization Library. In: *Large Data Analysis and Visualization (LDAV)*, 2011 IEEE Symposium on, pp. 89–96 (2011). DOI 10.1109/LDAV.2011.6092322
22. Habib, S., Pope, A., Finkel, H., Frontiere, N., Heitmann, K., Daniel, D., Fasel, P., Morozov, V., Zagaris, G., Peterka, T., et al.: HACC: Simulating Sky Surveys on State-of-the-Art Supercomputing Architectures. *New Astronomy* (2015)

23. Hart, P.E., Nilsson, N.J., Raphael, B.: A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics* **4**(2), 100–107 (1968). DOI 10.1109/TSSC.1968.300136
24. Humphrey, W., Dalke, A., Schulten, K.: VMD – Visual Molecular Dynamics. *Journal of Molecular Graphics* **14**, 33–38 (1996)
25. Li, M., Vazhkudai, S.S., Butt, A.R., Meng, F., Ma, X., Kim, Y., Engelmann, C., Shipman, G.: Functional partitioning to optimize end-to-end performance on many-core architectures. In: *Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis, SC '10*, pp. 1–12. IEEE Computer Society, Washington, DC, USA (2010). DOI 10.1109/SC.2010.28. URL <http://dx.doi.org/10.1109/SC.2010.28>
26. Liu, Q., Logan, J., Tian, Y., Abbasi, H., Podhorszki, N., Choi, J.Y., Klasky, S., Tchoua, R., Lofstead, J., Oldfield, R., Parashar, M., Samatova, N., Schwan, K., Shoshani, A., Wolf, M., Wu, K., Yu, W.: Hello ADIOS: the Challenges and Lessons of Developing Leadership Class I/O Frameworks. *Concurrency and Computation: Practice and Experience* **26**(7), 1453–1473 (2014). DOI 10.1002/cpe.3125. URL <http://dx.doi.org/10.1002/cpe.3125>
27. Mommessin, C., Dreher, M., Raffin, B., Peterka, T.: Automatic data filtering for in situ workflows. In: *2017 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 370–378. IEEE (2017)
28. Morozov, D., Peterka, T.: Block-Parallel Data Analysis with DIY2 (2016)
29. Morozov, D., Peterka, T.: Efficient Delaunay Tessellation through K-D Tree Decomposition. In: *Proceedings of SC16*. IEEE Press (2016)
30. Morton: A computer oriented geodetic data base and a new technique in file sequencing. Tech. Rep. Ottawa, Ontario, Canada (1966)
31. Peterka, T., Croubois, H., Li, N., Rangel, E., Cappello, F.: Self-adaptive density estimation of particle data. *SIAM Journal on Scientific Computing* **38**(5), S646–S666 (2016)
32. Peterka, T., Morozov, D., Phillips, C.: High-Performance Computation of Distributed-Memory Parallel 3D Voronoi and Delaunay Tessellation. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 997–1007. IEEE Press (2014)
33. Pronk, S., Pall, S., Schulz, R., Larsson, P., Bjelkmar, P., Apostolov, R., Shirts, M.R., Smith, J.C., Kasson, P.M., van der Spoel, D., Hess, B., Lindahl, E.: Gromacs 4.5: a high-throughput and highly parallel open source molecular simulation toolkit. *Bioinformatics* **29**(7), 845–854 (2013)
34. Whitlock, B., Favre, J.M., Meredith, J.S.: Parallel In Situ Coupling of Simulation with a Fully Featured Visualization System. In: *Proceedings of the 11th Eurographics Conference on Parallel Graphics and Visualization, EGPGV '11*, pp. 101–109. Eurographics Association, Aire-la-Ville, Switzerland, Switzerland (2011). DOI 10.2312/EGPGV/EGPGV11/101-109. URL <http://dx.doi.org/10.2312/EGPGV/EGPGV11/101-109>
35. Yu, H., Wang, C., Grout, R., Chen, J., Ma, K.L.: In situ visualization for large-scale combustion simulations. *Computer Graphics and Applications, IEEE* **30**(3), 45–57 (2010)
36. Zheng, F., Abbasi, H., Cao, J., Dayal, J., Schwan, K., Wolf, M., Klasky, S., Podhorszki, N.: In-situ i/o Processing: A Case for Location Flexibility. In: *Proceedings of the Sixth Workshop on Parallel Data Storage, PDSW '11*, pp. 37–42. ACM, New York, NY, USA (2011). DOI 10.1145/2159352.2159362. URL <http://doi.acm.org/10.1145/2159352.2159362>
37. Zheng, F., Zou, H., Eisenhauer, G., Schwan, K., Wolf, M., Dayal, J., Nguyen, T.A., Cao, J., Abbasi, H., Klasky, S., Podhorszki, N., Yu, H.: FlexIO: I/O Middleware for Location-Flexible Scientific Data Analytics. In: *Parallel Distributed Processing (IPDPS), 2013 IEEE 27th International Symposium on*, pp. 320–331 (2013). DOI 10.1109/IPDPS.2013.46